H2S-NN Revision: Progress Report

Letting the model keep disagreement, giving it a view of each node's neighbourhood, and testing the path mechanism directly

8 November 2026 (second report) Measurements: ACM dataset, NVIDIA RTX 4060 Laptop GPU (8 GB)

Overview

The previous report made the model fit in memory and coupled each meta-path face directly to every node on it. This stage asked what the model does with that information once it reaches a node, and whether the path mechanism does what the thesis claims. Two changes and two checks came out of it:

1. **A frequency-selective encoder.** The diffusion step could only smooth. A learned filter now lets each feature channel keep, damp or sharpen the disagreement between neighbours.
2. **Geometry that sees the neighbourhood.** The restriction maps, gates and path weights now read a summary of each node's typed neighbours, not only the node's own features. The submitted paper describes this term, but the code did not implement it.

Two checks support these changes (Section 3): the data pipeline against the official benchmark, and a synthetic test of the path mechanism in which we know which path instance holds the answer.

+0.3%	**+39%**	**Exact**	**1.0 vs 0.54**
time for the new filter (memory +0.04%)	time for the neighbourhood summary	match of our splits, labels and edges with the official benchmark	path test accuracy with correct vs learned path weights (chance 0.5)

No benchmark accuracy is reported. All runs so far use untuned default settings. Accuracy comparisons will run on the cluster under one protocol, with the same tuning budget for every method.

1. A frequency-selective encoder

Before

Propagation was K small diffusion steps. Each step pulled every node toward its neighbours, after transporting the neighbours with the restriction maps:

> $x \leftarrow (I - \eta A)^K h_0$ A = gated, normalised sheaf Laplacian of edges and path faces

A signal on the graph can be read as a mix of frequencies. A low-frequency signal is one where each node agrees with its transported neighbours. A high-frequency signal is one where it disagrees. The operator A measures this, with eigenvalues from 0 (full agreement) to 2 (strong disagreement). Each step multiplies a component at frequency λ by $(1 - \eta\lambda)$, so K steps multiply it by $(1 - \eta\lambda)^K$.

Why it is a problem

The old update can only smooth, and it smooths every feature channel in the same way. With the step size the model learned ($\eta \approx 0.45$) and $K = 4$:

- a component at $\lambda = 1$ keeps about 9% of its size;
- a component at $\lambda = 1.5$ keeps about 1%.

Under heterophily, the disagreement itself can carry the label. A paper whose subject is shared mostly by papers on a different topic is informative *because* it differs from them. The old update erases exactly this signal.

After

The output is a learned polynomial filter of the same operator:

$$x = \Sigma_{k=0..K}\ T_k(A - I) \cdot W_k\ h_0 \qquad \text{with} \quad \Sigma_k\ \|W_k\| \le 1$$

- T_k are the Chebyshev polynomials, a standard, numerically well-behaved basis.
- W_k are small d×d matrices, one per degree, that mix the feature channels.

Each channel therefore gets its own frequency response (Figure 1). The filter applies A K times, as the old K steps did, and adds only small per-node matrix products, so the memory savings of the previous report are kept.

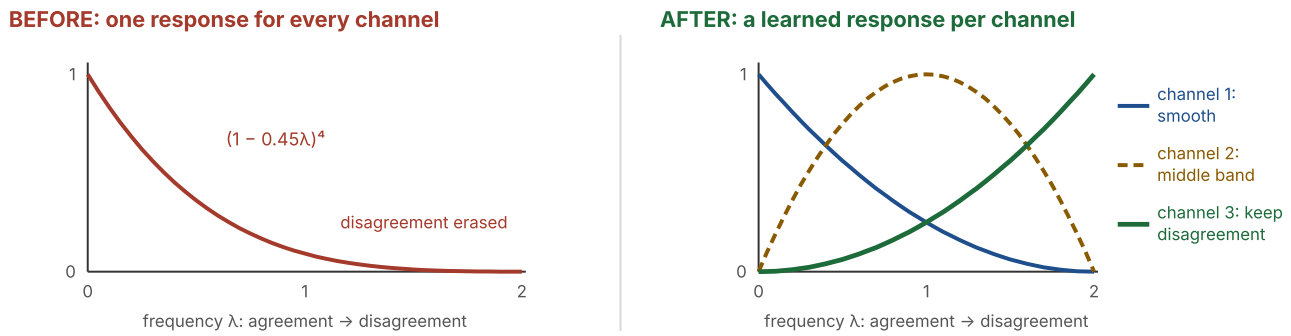


Figure 1. How strongly each frequency survives propagation. Before: one fixed curve for all channels, shown with the learned $\eta \approx 0.45$ and $K = 4$. After: each channel has its own curve. The three curves on the right are example shapes the filter can represent, not learned values.

Example. One channel can carry a paper's topic with a smoothing response, so papers that share authors move toward agreement. A second channel can use a high-pass response. A paper whose topic differs from the other papers on its subject then keeps that difference, instead of losing 99% of it as in the old update.

Properties we proved and checked numerically

- **It is stable for every parameter value.** The gated operator has a real spectrum in [0, 2), as the previous report showed. With the bound on the W_k, the filter's gain is at most 1, whatever values are learned.
- **The old update is a special case.** Setting every W_k to a fixed multiple of the identity gives $(I - \eta A)^K$ exactly. At initialisation the new model computes the same output as the old one, to within 10^{-10}.

- **The listen and broadcast gates keep their meaning through the whole filter.** A node whose listen gate is closed receives nothing from any other node, and a node whose broadcast gate is closed sends nothing. The corresponding blocks are exactly zero.
- **The node's own signal is never washed out.** The degree-0 term keeps the input, so adding degrees cannot make the output vanish. Deep sheaf diffusion models are known to suffer from that problem.

What this change gives up, and what it keeps

- **The energy reading is lost.** The old update could be read as a flow that lowers the sheaf energy at every step. The filter can favour disagreement, so this reading no longer holds and the paper will not claim it. Stability and the real spectrum are kept.
- **The stability bound limits what can be learned.** In a short smoke run, the unconstrained filter weights exceeded the bound within five epochs (sum 1.067) and were scaled back. Training pushes against the guarantee. That is the price of a guarantee that holds for every parameter value, and we will measure it rather than assume it is free.

Result

ACM, d = 32, one training step	Peak GPU memory	Time	Parameters
Old update	1.57 GB	1.47 s	532,649
Frequency-selective filter	**1.57 GB**	**1.48 s**	**537,768**

Both with the neighbourhood summary of Section 2. Mean of 3 runs after a warm-up. The cost limit set in advance was 25% time and 10% memory; the measured cost is 0.3% and 0.04%.

Status. That frequency selectivity helps under heterophily is a hypothesis we have not yet observed. On the cluster the filter will be compared with the old update, which is nested inside it, on the heterophilic datasets (Freebase, IMDB and the H2GB sets).

2. Geometry that sees the neighbourhood

Before

The restriction maps, the listen and broadcast gates and the path weights were all computed from a node's own input features h_0. The submitted paper includes a neighbourhood-context term for the maps, but the code did not implement it, so paper and code disagreed.

Why it is a problem

Two nodes with similar features can play different roles in the graph. If the geometry sees only the node itself, it cannot tell them apart, and both receive the same maps and gates.

After

Each node gets a typed neighbourhood summary: for each relation type, the average input features of its neighbours through that relation. The geometry reads both parts:

```
c_v  =  [ mean of h_0 over v's authors ,  mean over v's subjects ,  … ]        (one
average per relation type)
maps, gates, path weights  =  functions of  [ h_0v , c_v ]
```

The summary is computed **once**, before propagation. Other sheaf models (NSD, CSNN, DNSD) recompute their maps from the propagated states at every layer, which creates a new sheaf at each layer. Computing the summary once keeps a single fixed sheaf. The guarantees of Section 1 and the mini-batch argument of the previous report therefore still hold.



Figure 2. Two papers with similar abstracts. Before, they receive the same maps and gates. After, the summary of their authors separates them.

Example. P_1 and P_2 have nearly identical abstracts. P_1's authors publish mostly in databases and P_2's mostly in machine learning, so their summaries differ. P_2 can then learn, for instance, to listen less to a subject node that is shared mainly with database papers.

Properties we proved and checked numerically

- **Nothing else changes when the summary is switched off.** The model then reproduces the previous model's outputs exactly.
- **The summary reaches the geometry, and only where it should.** Changing one neighbour's features changes that node's maps and gates when the summary is on, and changes nothing when it is off. The summary uses only the stored facts and respects each relation's direction.

What this change gives up, and what it keeps

- **The geometry does not follow the propagated states.** It is fixed before propagation, by design: this keeps one sheaf and all its guarantees. Models that recompute their maps at every layer can adapt their geometry during propagation; ours cannot.
- **It is the most expensive change of this stage** (Result, below).
- **It overlaps partly with the path faces.** The summary gives each node a view two hops away: an author's summary contains its papers, and a paper reads that summary through its author. Section 3.2 shows that this alone solved a task we had designed to test path faces. This is not a defect, but it raises the bar for the thesis. Path faces must add information beyond edges *plus* neighbourhood summaries. We therefore keep the summary on in every arm of the path comparison, which is the stricter test, and the paper will say why.

Result

ACM, d = 32, one training step	Peak GPU memory	Time	Parameters
Geometry from the node alone	1.49 GB	1.06 s	307,880
Geometry from node and neighbourhood	**1.57 GB**	**1.48 s**	**537,768**

Both with the filter of Section 1. Mean of 3 runs after a warm-up.

The summary costs +39% time, +5.4% memory and about 230,000 parameters. The parameters grow because every network that computes the geometry now reads a wider input.

Status. Whether the summary improves accuracy, and whether that is worth 39% more time, will be tested on the cluster: geometry from the node alone vs node and neighbourhood.

3. Further checks: the data pipeline and the path mechanism

3.1 The data pipeline matches the benchmark

Before changing the model, we checked that our data matches the HGB benchmark (ACM, DBLP, IMDB) exactly. We read the official files with HGB's own loader and compared them item by item:

- **Splits:** our test nodes are exactly HGB's test nodes. Our training and validation nodes together are exactly HGB's labelled set, with HGB's 20% held out for validation.
- **Labels:** all 3,025, 4,057 and 4,573 labels agree, including IMDB's multi-label vectors.
- **Edges:** our graph stores one edge per fact. Expanded back to both directions, it reproduces HGB's stored edge list exactly.

We could not reproduce HetSheaf's published rerun of R-GCN, for three reasons:

- their selected configuration came from a 150-run search that is not published;
- their paper states HGB's 24/6/70 split, but their code uses 27/3/70;
- their code duplicates every relation, giving ACM 14 relation types instead of 8 and twice the edges.

Their own pipeline also misses the published number with the configuration in their repository.

Decision. We use HGB's official split. Every baseline, HetSheaf included, will be rerun under our protocol. Published numbers will appear only in a clearly labelled note, not in the comparison tables.

3.2 A test of the path mechanism with a known answer

Our thesis is that a meta-path instance carries information through its intermediate node and the order of its relations, and that this information is lost when the instance is reduced to a relation between its two endpoints. Before running large comparisons, we wanted to see the mechanism do this on a graph where we know which instance holds the answer.

First design: the label is a co-paper's class, reached through the lead author

Each paper has a lead author and a second author, and each author has one other paper, a co-paper. The paper's label is the class of the co-paper reached through its lead author.

A model *without* path faces scored 0.996, so the task leaked. The cause was the neighbourhood summary. The lead author's summary already contains its co-paper's class, and the paper reads that summary through an ordinary edge. With the summary set to zero, the same trained model fell to chance (0.488). This is the overlap described in Section 2.

Second design: only the whole instance determines the label

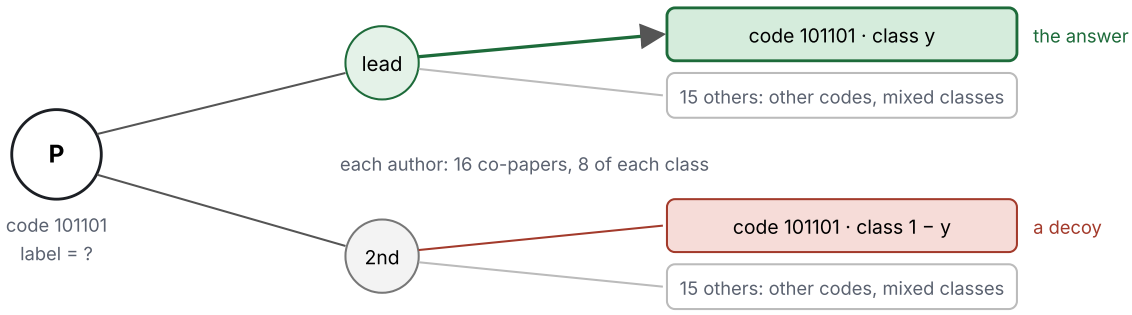


Figure 3. The second test. P's label is the class of the co-paper that is reached through the **lead** author **and** carries P's code. The second author also has a co-paper with P's code, of the opposite class. Neither the author alone, nor the code alone, nor the two endpoints alone give the answer; only the whole instance P – lead author – co-paper does.

The graph has 17,000 papers, 2,000 authors and 272,000 path instances. Each paper carries a 6-bit code, and each author has 16 co-papers, balanced by class. Of the 1,000 labelled papers, 500 are used for training. We fixed the pass criteria before any run:

- the full model must reach at least 0.90 on every seed;
- every control must stay at or below 0.60 (chance is 0.5).

Model (3 seeds)	Test accuracy	Reading
Controls: edges only (no path faces), endpoint relations only, own features only	0.48 – 0.56	at chance: the task does not leak
Full model, path weights learned	**0.50 / 0.58 / 0.54**	**at chance: fails the criterion**
Path faces with equal weights	0.58 / 0.56 / 0.53	at chance, as expected
Full model, correct path weights given (diagnostic)	1.00 / 0.996 / 1.00	solved

What the result shows

- **The rest of the model is not the bottleneck.** When we set the path weights to the correct answer and train everything else as usual, the model solves the task. The transports along the path, the filter and the classifier carry the information.
- **Learning the path weight is the bottleneck.**
  - The learned weights did separate lead-author instances from second-author ones. That depends on a property of a single node.

- They never separated instances whose codes match from those whose codes do not. That needs a comparison between two nodes on the path.
- The weight network sees all three nodes and has a hidden layer, so it can represent this rule in principle. It did not learn it from 500 papers in 200 epochs at default settings.
- Its weights did not differ between training and test papers, so it was not memorising instances.

Status. The test neither confirms nor refutes the thesis on real data. The task is deliberately hard (an exact 6-bit match learned from 500 examples), and we did not tune for it, because tuning on a known-answer task would make a pass meaningless. It does show where the risk sits. Path instances add information beyond graph diffusion only when they are weighted by their content, and part of that content is a relationship between nodes on the path. The weight function's ability to learn such comparisons is therefore central to the thesis. We carry this question, and both test tasks, into the next stage.

4. Cost

ACM, d = 32, one training step	Peak GPU memory	Time	Parameters
Submitted model	does not fit in 8 GB	—	1.62M
After lighter maps and direct path coupling (previous report)	2.5 GB	1.77 s	0.31M
After fact storage, path sampling and gates (previous report)	1.49 GB	1.06 s	0.30M
+ frequency-selective filter (Section 1)	1.49 GB	1.06 s	0.31M
+ neighbourhood summary (Section 2): current model	**1.57 GB**	**1.48 s**	**0.54M**

Peak GPU memory and time for one training step (forward and backward). The last three rows come from one measurement session (3 runs after a warm-up), the second row from the earlier session (5 runs).

	Previous model	Current model
Propagation	K diffusion steps: $O(K \cdot (M + F \cdot L) \cdot d^2)$	same K applications of A, plus channel mixing $O(N \cdot K \cdot d^2)$
Neighbourhood summary	—	one pass over the facts: $O(M \cdot d)$
Input to the geometry networks	d numbers per node	grows with the number of relation types; the first layer is evaluated once per node, not once per edge

N = number of nodes, M = number of facts, F = number of path instances, L = path length, K = filter degree.

The current model fits where the submitted one did not fit at all, and has about three times fewer parameters. On this GPU the memory saving can only be bounded from below, at 4.8× or more. Confirming the 10× target of the revision plan needs a measurement of the submitted model on a larger GPU. Almost all of this stage's added

cost comes from the neighbourhood summary, not from the filter. Neither change alters how cost grows with graph size.

5. Next steps

1. **The new path mechanism.** It samples walks without a fixed meta-path list, gives each instance a learned weight, and reports meta-path importance afterwards instead of selecting meta-paths in advance. Before building it we will decide whether the weight function needs an explicit comparison between nodes on the path. Both known-answer tasks will be rerun on it. The first will run with the neighbourhood summary switched off, so that it tests the path weights rather than the summary.
2. **Comparisons on the cluster.** Each is fixed before running and uses the same tuning budget, with 10 seeds and paired tests:
   - the filter vs the old update, on the heterophilic datasets;
   - geometry with vs without the neighbourhood summary;
   - learned path weights vs endpoint-only weights vs no path faces, with the summary on in every arm.
3. **All baselines rerun** under the same protocol and split.